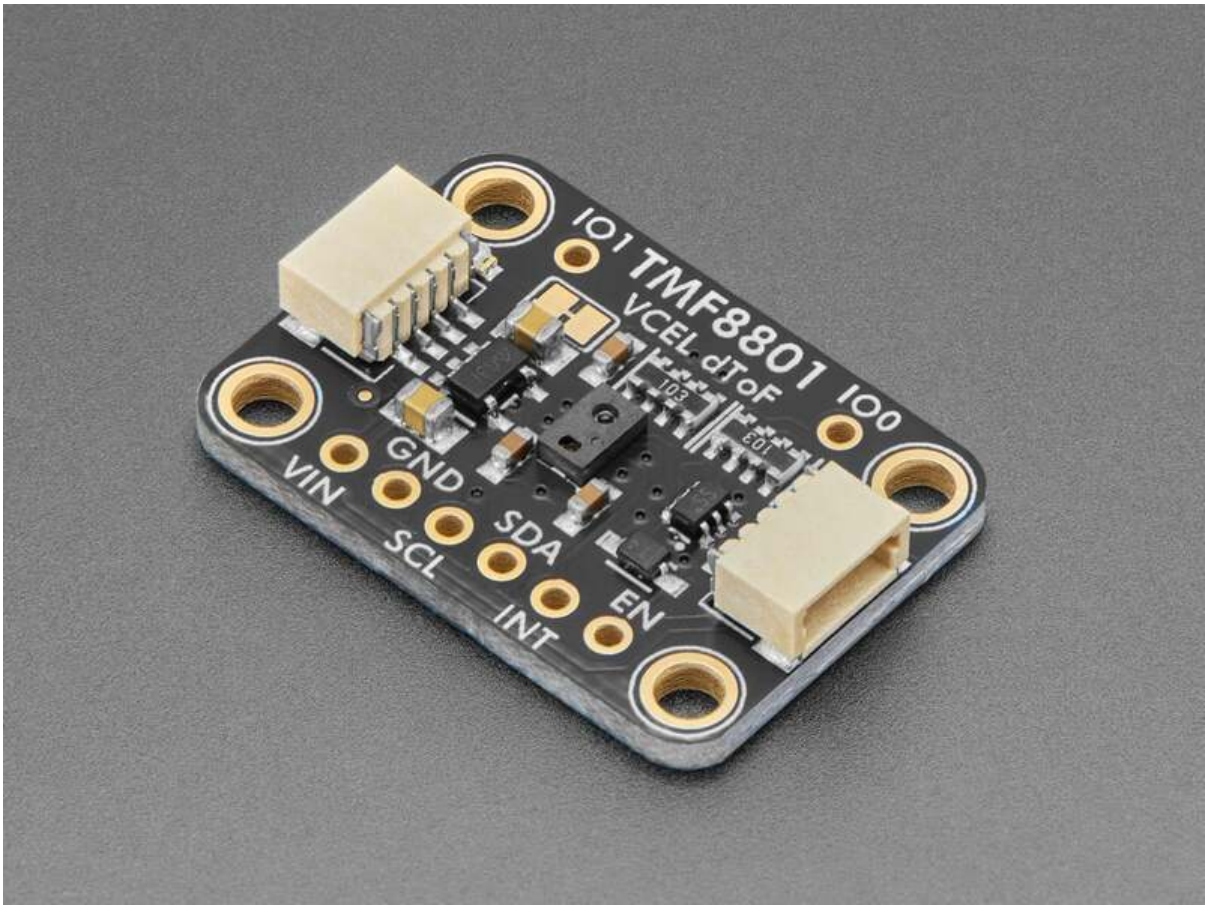
Adafruit TMF8801 Time of Flight Distance Sensor

Created by Liz Clark



https://learn.adafruit.com/adafruit-tmf8801-time-of-flight-distance-sensor

Last updated on 2026-09-17 02:21:18 PM UTC

Table of Contents

Overview 3

Pinouts 6

- Power Pins
- I2C Logic Pins
- Other Pins
- Power LED and LED Jumper

CircuitPython and Python 7

- CircuitPython Microcontroller Wiring
- Python Computer Wiring
- Python Installation of TMF8801 Library
- CircuitPython Usage
- Python Usage
- Example Code

Python Docs 11

Arduino 12

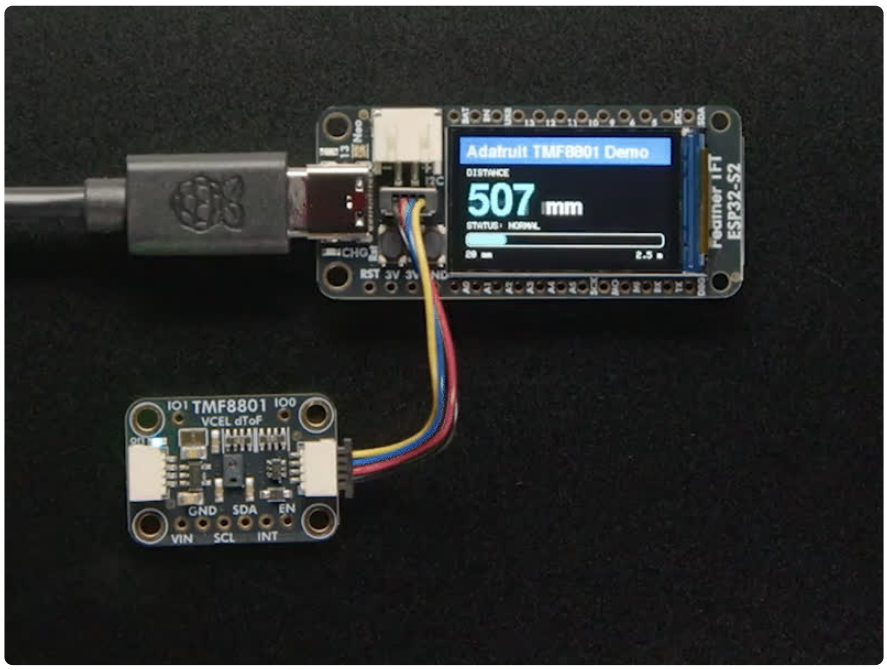
- Wiring
- Library Installation
- Example Code

Arduino Docs 16

Downloads 16

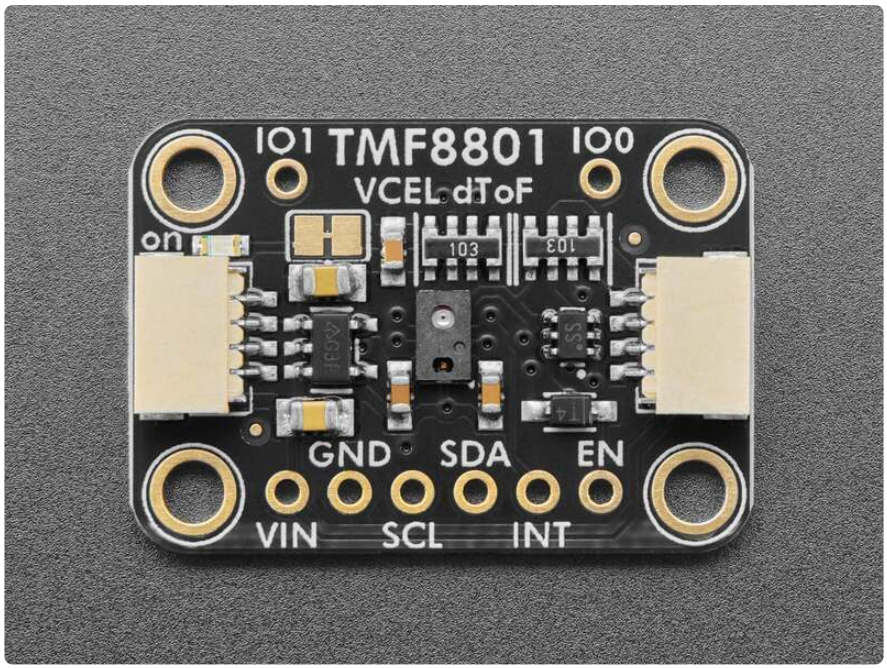
- Files
- Schematic and Fab Print
- 3D Model

Overview



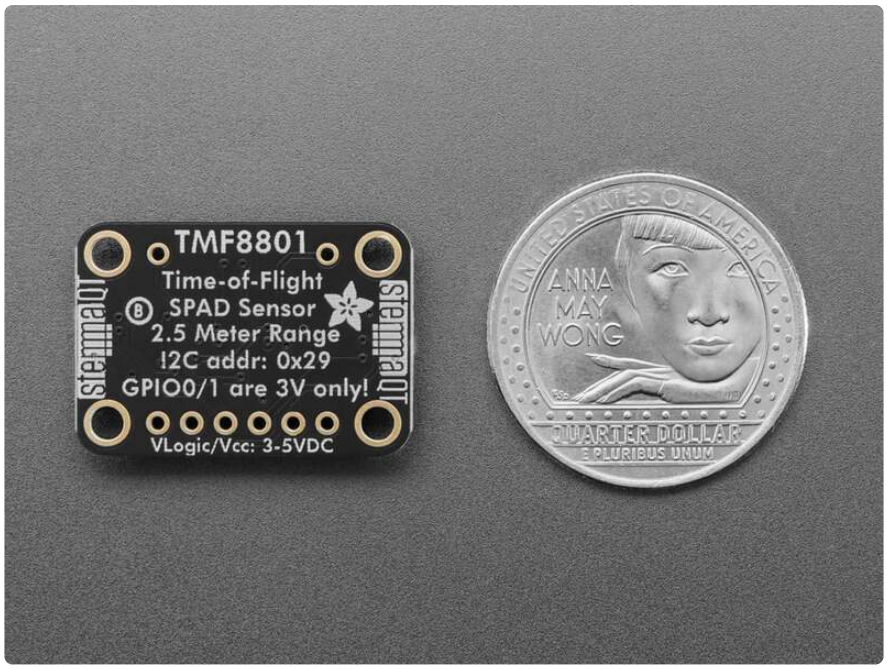
The **Adafruit TMF8801 Time of Flight Sensor** is a great Time of Flight distance sensor from ams Osram in the TMF series of chips. The TMF8801 in particular can do 20mm to 2.5 meters at 33 Hz update rate.

The sensor contains a very tiny invisible laser source and a matching sensor. The TMF8801 can detect the "time of flight", or how long the light has taken to bounce back to the sensor. Since it uses a very narrow light source, it is good for determining distance of only the surface directly in front of it. Unlike sonars that bounce ultrasonic waves, the 'cone' of sensing is very narrow. Unlike IR distance sensors that try to measure the amount of light bounced, the TMF is much more precise and doesn't have linearity problems or 'double imaging' where you can't tell if an object is very far or very close.

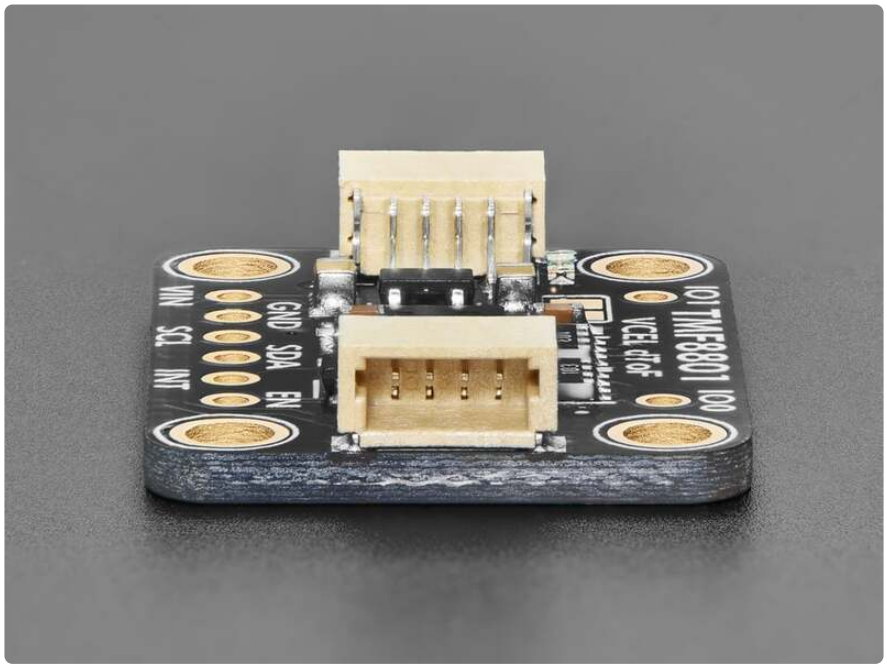


One nice thing about this sensor is that while it does require a firmware 'patch' on boot, the SRAM usage is also much lighter, so it fits easily in small memory boards like the ATmega328. You can use it on many 8-bit microcontroller devices.

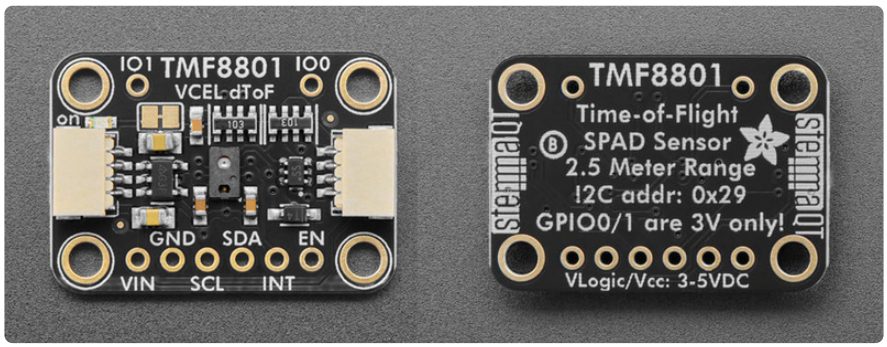
The sensor is small and easy to use in any robotics or interactive project. Since it needs 3.3V power and logic, we put the little fellow on a breakout board with a regulator and level shifting. You can use it with any 3-5V power or logic microcontroller with no worries. Works great with the **3.3V logic level of a Feather, or the 5V level of a ATmega**. This breakout is ready to work with most common microcontrollers or single-board computers (SBCs). And since it speaks I2C, you can easily connect it up with two data wires plus power and ground.



As if that weren't enough, we've also added SparkFun qwiic () compatible STEMMA QT **()** connectors for the I2C bus **so you don't even need to solder.** Just wire up to your favorite micro with a plug-and-play cable to get ToF data ASAP. For a no-solder experience, just wire up to your favorite micro using a STEMMA QT adapter cable. () The Stemma QT connectors also mean the TMF8801 can be used with our various associated accessories. () QT Cable is not included, but we have a variety in the shop ()



Pinouts



The default I2C address is **0x29**.

Power Pins

- **VIN** - This is the power pin. To power the board, give it the same power as the logic level of your microcontroller - e.g. for a 5V micro like Arduino, use 5V.
- **GND** - This is common ground for power and logic.

I2C Logic Pins

- **SCL** - I2C clock pin, connect to your microcontroller's I2C clock line. This pin can use 3-5V logic, and there's a **10K pullup** on this pin.
- **SDA** - I2C data pin, connect to your microcontroller's I2C data line. This pin can use 3-5V logic, and there's a **10K pullup** on this pin.
- STEMMA QT () - These connectors allow you to connectors to dev boards with **STEMMA QT** connectors or to other things with various associated accessories ()

Other Pins

- INT - the interrupt pin. It is an open-drain interrupt.
- EN - the enable pin. The pin is active high. If you pull the pin low, it turns off the TMF8801 and all memory content is lost
- IO0 and IO1 - these are general purpose input and output pins. They default to output low. They are 3V tolerant only!

Text emphasized with a yellow exclamation:IO0 and IO1 are 3V only!

Power LED and LED Jumper

- **Power LED -** In the upper left corner, above the STEMMA connector, on the front of the board, is the power LED, labeled **on**. It is a green LED.
- **LED jumper** - This jumper is located on the front of the board, the the right of the LED. Cut the trace on this jumper to cut power to the "on" LED.

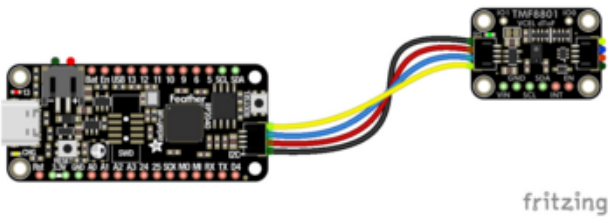
CircuitPython and Python

It's easy to use the **TMF8801** with Python or CircuitPython, and the Adafruit_CircuitPython_TMF8801 () module. This module allows you to easily write Python code to interface with the sensor.

You can use this driver with any CircuitPython microcontroller board or with a computer that has GPIO and Python thanks to Adafruit_Blinka, our CircuitPython-for-Python compatibility library ().

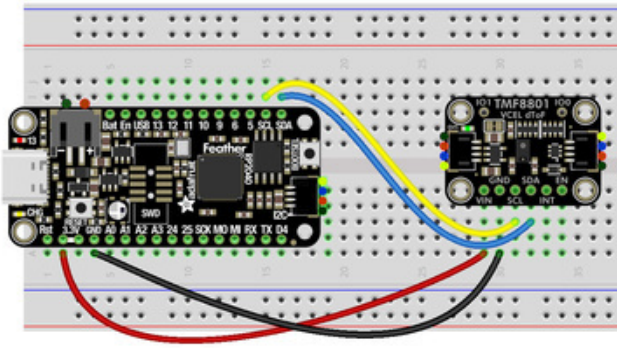
CircuitPython Microcontroller Wiring

First wire up the breakout to your board exactly as follows. The following is the breakout wired to a Feather RP2040 using the STEMMA connector:



Board STEMMA 3V to **breakout STEMMA VIN (red wire)**
Board STEMMA GND to **breakout STEMMA GND (black wire)**
Board STEMMA SCL to **breakout STEMMA SCL (yellow wire)**
Board STEMMA SDA to **breakout STEMMA SDA (blue wire)**

The following is the breakout wired to a Feather RP2040 using a solderless breadboard:

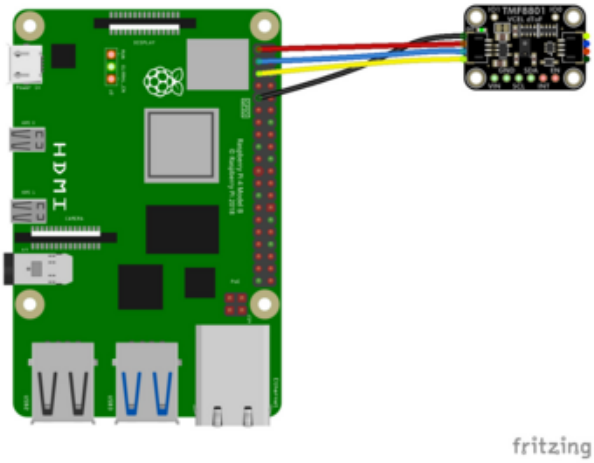


Board 3V to **breakout VIN (red wire)**
Board GND to **breakout GND (black wire)**
Board SCL to **breakout SCL (yellow wire)**
Board SDA to **breakout SDA (blue wire)**

Python Computer Wiring

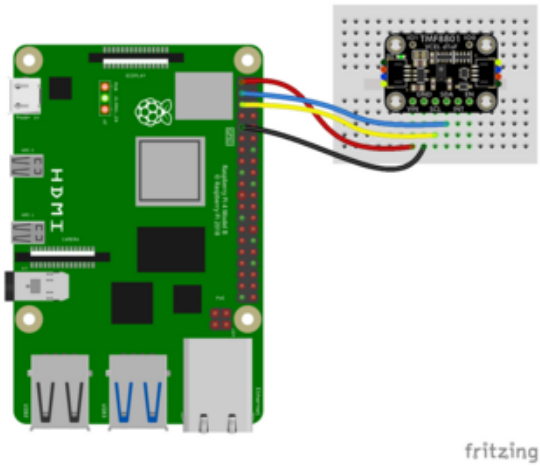
Since there are dozens of Linux computers/boards you can use, we will show wiring for Raspberry Pi. For other platforms, [please visit the guide for CircuitPython on Linux to see whether your platform is supported]() ().

Here's the Raspberry Pi wired with I2C using the STEMMA connector:



Pi 3V to **breakout STEMMA VIN (red wire)**
Pi GND to **breakout STEMMA GND (black wire)**
Pi SCL to **breakout STEMMA SCL (yellow wire)**
Pi SDA to **breakout STEMMA SDA (blue wire)**

Here's the Raspberry Pi wired with I2C using a solderless breadboard:



Pi 3V to **breakout VIN (red wire)**
Pi GND to **breakout GND (black wire)**
Pi SCL to **breakout SCL (yellow wire)**
Pi SDA to **breakout SDA (blue wire)**

Python Installation of TMF8801 Library

You'll need to install the **Adafruit_Blinka** library that provides the CircuitPython support in Python. This may also require enabling I2C on your platform and verifying you are running Python 3. Since each platform is a little different, and Linux changes often, please visit the CircuitPython on Linux guide to get your computer ready ()!

Once that's done, from your command line run the following command:

- `pip3 install adafruit-circuitpython-tmf8801`

If your default Python is version 3 you may need to run 'pip' instead. Just make sure you aren't trying to use CircuitPython on Python 2.x, it isn't supported!

CircuitPython Usage

To use with CircuitPython, you need to first install the **Adafruit_CircuitPython_TMF8801** library, and its dependencies, into the **lib** folder on your **CIRCUITPY** drive. Then you need to update **code.py** with the example script.

Thankfully, we can do this in one go. In the example below, click the **Download Project Bundle** button below to download the necessary libraries and the **code.py** file in a zip file. Extract the contents of the zip file, and copy the **entire lib folder** and the **code.py** file to your **CIRCUITPY** drive.

Your **CIRCUITPY/lib** folder should contain the following folders:

- **adafruit_bus_device/**
- **adafruit_register/**
- **adafruit_tmf8801/**



Python Usage

Once you have the library `pip3` installed on your computer, copy or download the following example to your computer, and run the following, replacing **code.py** with whatever you named the file:

```
python3 code.py
```

Example Code

If running CircuitPython: Once everything is saved to the **CIRCUITPY** drive, connect to the serial console () to see the data printed out!

If running Python: The console output will appear wherever you are running Python.

```python
# SPDX-FileCopyrightText: Copyright (c) 2026 Liz Clark for Adafruit Industries
#
# SPDX-License-Identifier: MIT

import board

from adafruit_tmf8801 import tmf8801

i2c = board.I2C()

# This uploads about 11.6 kB of firmware to the sensor
sensor = tmf8801.TMF8801(i2c)

print(f"Firmware version: {'.'.join(str(part) for part in sensor.firmware_version)}")
print(f"Serial number: {sensor.serial_number:#06x}")

sensor.start_measuring()

while True:
    if sensor.data_ready:
        distance = sensor.distance
        if distance is not None:
            print(f"Distance: {distance} mm")
```

https://github.com/adafruit/Adafruit_CircuitPython_TMF8801/blob/main/examples/tmf8801_simpletest.py

You'll see the TMF8801 recognized over I2C by printing its firmware version and serial number. Then, distance measurements will begin printing whenever data is ready from the sensor.

```
Auto-reload is on. Simply save files over USB to run them or enter REPL to disable.
code.py output:
Firmware version: 3.0.22
Serial number: 0x49c8
Distance: 49 mm
Distance: 80 mm
Distance: 98 mm
Distance: 174 mm
Distance: 234 mm
Distance: 323 mm
Distance: 512 mm
Distance: 39 mm
Distance: 218 mm
Distance: 524 mm
Distance: 682 mm
```

Python Docs

Python Docs ()

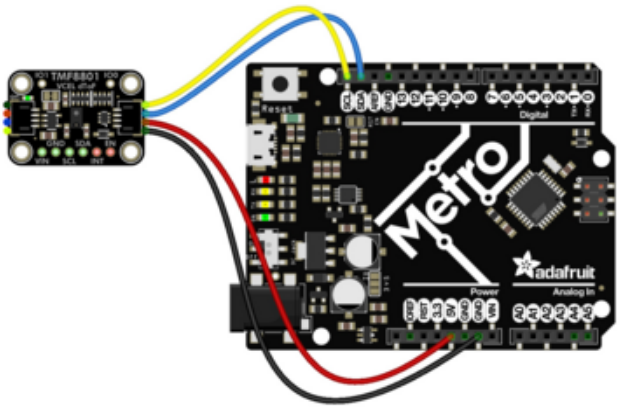
Arduino

Using the TMF8801 breakout with Arduino involves wiring up the breakout to your Arduino-compatible microcontroller, installing the Adafruit_TMF8801 () library, and running the provided example code.

Wiring

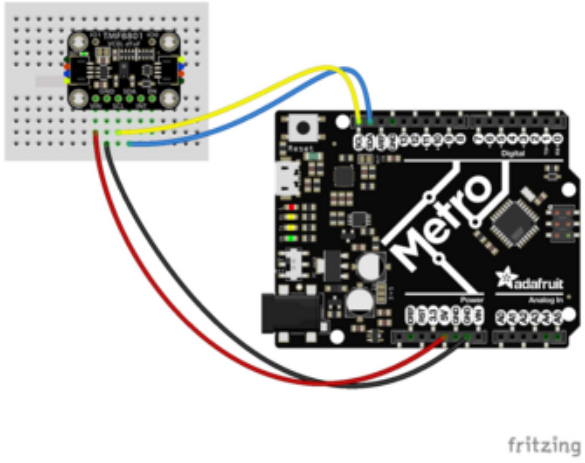
Wire as shown for a **5V** board like an Uno. If you are using a **3V** board, like an Adafruit Feather, wire the board's 3V pin to the breakout VIN.

Here is an Adafruit Metro wired up to the breakout using the STEMMA QT connector:



Board 5V to **breakout VIN (red wire)**
Board GND to **breakout GND (black wire)**
Board SCL to **breakout SCL (yellow wire)**
Board SDA to **breakout SDA (blue wire)**

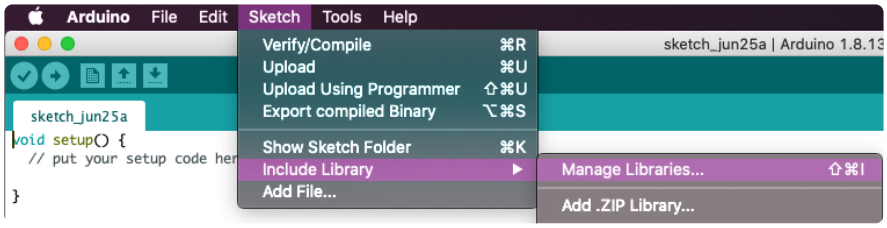
Here is an Adafruit Metro wired up using a solderless breadboard:



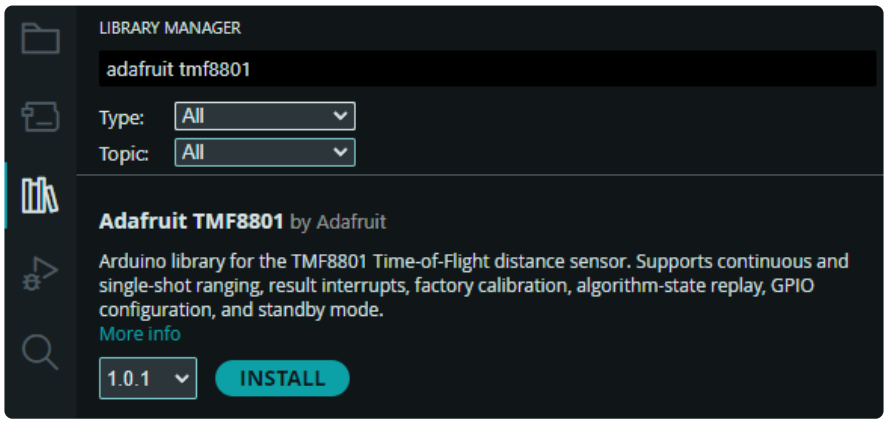
Board 5V to **breakout VIN (red wire)**
Board GND to **breakout GND (black wire)**
Board SCL to **breakout SCL (yellow wire)**
Board SDA to **breakout SDA (blue wire)**

Library Installation

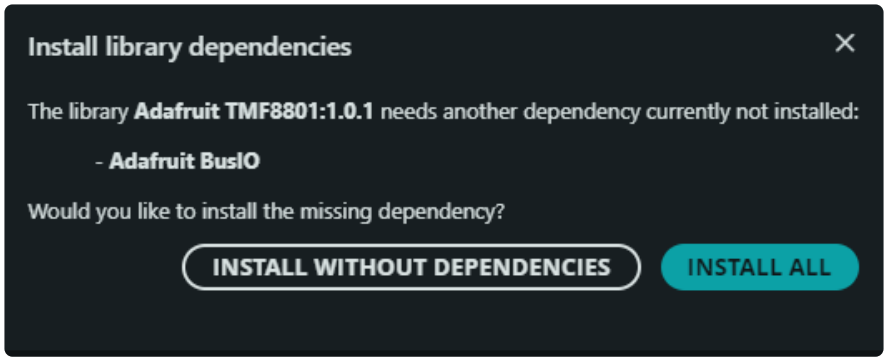
You can install the **Adafruit_TMF8801** library for Arduino using the Library Manager in the Arduino IDE.



Click the **Manage Libraries ...** menu item, search for **Adafruit_TMF8801**, and select the **Adafruit TMF8801** library:



If asked about dependencies, click "Install all".



If the "Dependencies" window does not come up, then you already have the dependencies installed.

Text emphasized with a yellow exclamation:

If the dependencies are already installed, you must make sure you update them through the Arduino Library Manager before loading the example!

Example Code

```
/*!
 * @file tmf8801_simpletest.ino
 *
 * Basic continuous-ranging example for the Adafruit TMF8801.
 *
 * Written by Limor Fried/Ladyada for Adafruit Industries.
 * MIT license, all text above must be included in any redistribution.
 */

#include <Adafruit_TMF8801.h>

Adafruit_TMF8801 tmf8801;

void setup() {
  Serial.begin(115200);
  while (!Serial) {
    delay(10);
  }

  Serial.println(F("Adafruit TMF8801 test"));

  if (!tmf8801.begin()) {
    Serial.println(F("Could not find a valid TMF8801 sensor, check wiring!"));
    while (1) {
      delay(10);
    }
  }
  Serial.println(F("TMF8801 found"));

  uint8_t major = 0;
  uint8_t minor = 0;
  uint8_t patch = 0;
  tmf8801.getVersion(&major, &minor, &patch);
  Serial.print(F("Firmware version: "));
  Serial.print(major);
  Serial.print(F("."));
  Serial.print(minor);
  Serial.print(F("."));
  Serial.println(patch);

  tmf8801.setIterations(900);      // 900,000 integration iterations
```

```
  tmf8801.setRepetitionPeriod(33); // About 30 measurements per second

  if (!tmf8801.startMeasuring()) {
    Serial.println(F("Could not start measurements"));
    while (1) {
      delay(10);
    }
  }
}

void loop() {
  if (tmf8801.dataReady()) {
    tmf8801_result_t result;
    if (tmf8801.readResult(&result)) {
      if (result.reliability == 0) {
        Serial.println(F("No object detected"));
      } else {
        Serial.print(F("Distance: "));
        Serial.print(result.distance_mm);
        Serial.print(F(" mm   Reliability: "));
        printReliability(result.reliability);
        Serial.print(F("   Status: "));
        printMeasurementStatus(result.status);
        Serial.println();
      }
    }
  }
}

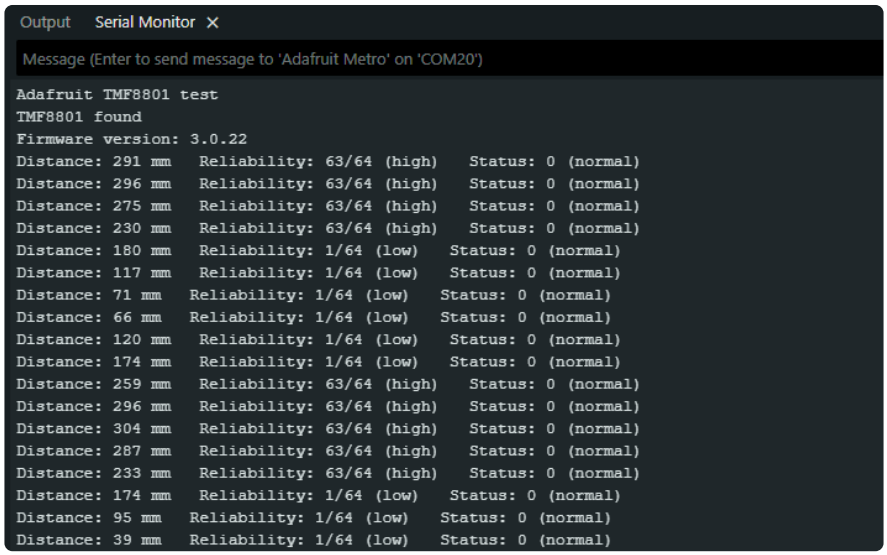
void printReliability(uint8_t reliability) {
  Serial.print(reliability);
  Serial.print(F("/"));
  Serial.print(TMF8801_RESULT_RELIABILITY_LEVELS);
  Serial.print(F(" ("));
  if (reliability >= TMF8801_RESULT_RELIABILITY_HIGH_MIN) {
    Serial.print(F("high"));
  } else {
    Serial.print(F("low"));
  }
  Serial.print(F(")"));
}

void printMeasurementStatus(tmf8801_measurement_status_t status) {
  Serial.print((uint8_t)status);
  Serial.print(F(" ("));
  switch (status) {
  case TMF8801_MEASUREMENT_NOT_INTERRUPTED:
    Serial.print(F("normal"));
    break;
  case TMF8801_MEASUREMENT_INTERRUPTED_BY_GPIO:
    Serial.print(F("interrupted by GPIO"));
    break;
  case TMF8801_MEASUREMENT_STATUS_RESERVED_1:
  case TMF8801_MEASUREMENT_STATUS_RESERVED_3:
    Serial.print(F("reserved"));
    break;
```

```
    default:
      Serial.print(F("unknown"));
      break;
    }
    Serial.print(F(")"));
  }
```

https://github.com/adafruit/Adafruit_TMF8801/blob/main/examples/tmf8801_simpletest/tmf8801_simpletest.ino

Upload the sketch to your board and open up the Serial Monitor (**Tools -> Serial Monitor**) at 115200 baud. You'll see the TMF8801 recognized over I2C. Then, the distance in mm, reliability of the measurement and status will be printed whenever data is ready.



Arduino Docs

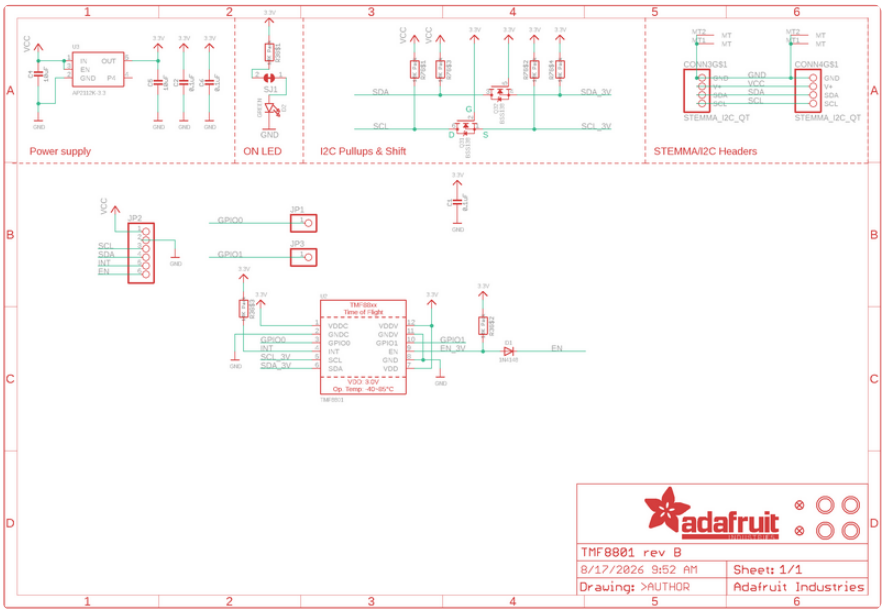
Arduino Docs ()

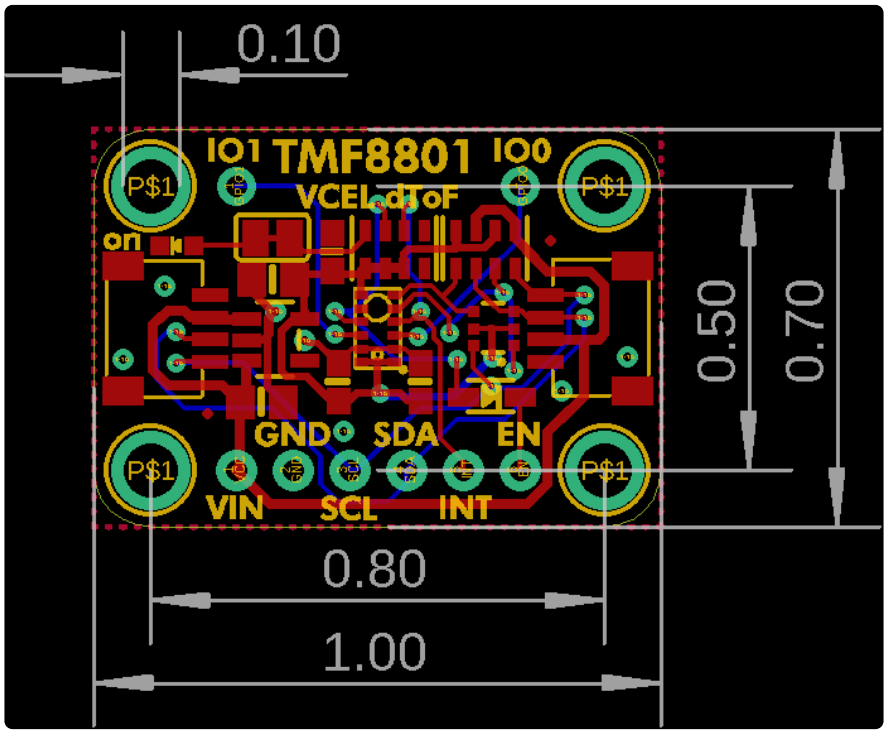
Downloads

Files

- TMF8801 Datasheet (http://adafru.it/652265228801)
- EagleCAD PCB Files on GitHub ()
- 3D models on GitHub ()
- Fritzing object in the Adafruit Fritzing Library ()

Schematic and Fab Print





3D Model

